

Tensor Cores, Numeric Formats, and Optimized GPU Libraries

This guide connects GPU number formats to the hardware and software paths that use them. The central operation is a matrix multiply-accumulate:

$$D = A \times B + C$$

In practice, use optimized libraries such as **cuBLAS/cuBLASLt** for matrix multiplication, **cuDNN** for neural-network primitives, and **TensorRT** for inference before writing WMMA or MMA code. Libraries already handle tiling, shared-memory movement, alignment, architecture selection, and many edge cases. WMMA and MMA are appropriate when implementing an operation that a library cannot express efficiently, especially a fused custom kernel.

Precision map

Format	Total bits	Sign	Exponent	Explicit fraction	Approximate strength	Typical GPU role
FP32	32	1	8	23	Good range and precision	General compute, reference, accumulation
FP16	16	1	5	10	Better precision than BF16, narrow range	Tensor Core training/inference inputs
BF16	16	1	8	7	FP32-like range, coarse precision	Stable mixed-precision training inputs
TF32	19 effective input bits	1	8	10	FP32 range, FP16-like significand precision	Fast FP32 matrix math on Tensor Cores
INT8	8	—	—	—	Exact integers in a small range	Quantized inference and integer dot products

A floating-point significand has an implicit leading bit for normal values. Thus FP32 has 24 bits of normal-number precision, FP16 has 11, BF16 has 8, and TF32 has about 11. TF32 is a Tensor Core computation format, not normally a C++ storage type.

FP32

1. Overview

Definition

FP32, or IEEE 754 binary32, is a 32-bit floating-point format containing one sign bit, eight exponent bits, and 23 stored fraction bits. For a normal finite value:

$$(-1)^{sign} \times 1.fraction \times 2^{exponent-127}$$

It represents a wide range of magnitudes, fractional values, signed zero, infinity, NaN, and subnormal numbers.

Why it matters

FP32 is the default numerical baseline for much CUDA code. It usually provides enough accuracy for graphics, simulation, machine learning, and signal processing while consuming half the storage of FP64. It is also a common accumulator type when FP16, BF16, TF32, or INT8 supplies the matrix inputs.

Where it is used

- CUDA `float` kernels and conventional CUDA cores
- cuBLAS SGEMM and FP32 accumulation modes
- Neural-network master weights, reductions, normalization, and reference runs
- Image processing, scientific codes, and simulations whose error tolerance fits FP32
- Tensor Core TF32 operations whose API inputs and outputs remain FP32

Why interviewers ask

FP32 tests whether a candidate understands representation error, range versus precision, non-associativity, accumulation, and why changing a data type changes performance as well as correctness.

2. Core Idea

Intuition and analogy

Scientific notation stores a sign, significant digits, and a power of ten. FP32 does the same in base two. Think of the exponent as selecting a zoom level and the significand as the number of tick marks visible at that zoom. A large exponent gives range; more fraction bits give fine resolution.

Small example

Around `1.0`, adjacent FP32 values are separated by approximately 2^{-23} . Around 2^{20} , the spacing is about $2^{(20-23)} = 0.125$. Precision is therefore **relative**, not a fixed number of decimal places.

```
float x = 16777216.0f; // 2^24
float y = x + 1.0f;
// y == x: at this magnitude FP32 cannot represent the intervening integer.
```

Step by step

1. The exact real result is computed conceptually.

2. Hardware rounds it to the nearest representable FP32 value, normally ties-to-even.
3. Every arithmetic operation can introduce rounding.
4. A long reduction can accumulate those errors.
5. Fused multiply-add (FMA) computes $a*b+c$ with one final rounding, often improving both speed and accuracy.

3. Important Subtopics

Representation, ULP, and machine epsilon

An **ULP** is the gap between neighboring representable values at a given magnitude. `FLT_EPSILON` is the gap from `1.0f` to the next larger FP32 value, approximately `1.19209e-7`. Interviewers often expect the distinction between relative precision and absolute error.

Normal and subnormal values

Normal values use an implicit leading `1`. Subnormals use exponent field zero and no implicit leading `1`, enabling gradual underflow below the smallest normal magnitude at reduced precision. Some performance modes flush subnormals to zero; do not assume every compilation mode preserves them.

NaN and infinity

Division by zero can produce infinity; invalid operations can produce NaN. NaN is unordered: comparisons such as `x == NaN` are false. Detect it with `isnan`. A single NaN can contaminate a reduction.

Non-associativity

Floating-point addition is not associative:

```
(a + b) + c may differ from a + (b + c)
```

Parallel reductions change grouping, so a GPU result can differ slightly from a serial CPU result without either being incorrect.

Accumulation strategy

Pairwise/tree reductions usually have smaller error growth than a long sequential sum. Kahan summation can improve accuracy but adds instructions and dependencies. Interview angle: select it only when the error requirement justifies the throughput cost.

4. Real-World Example

Consider computing a neural-network layer with FP16 weights and activations. Tensor Cores multiply the FP16 inputs but accumulate partial sums in FP32. The wider accumulator reduces loss when thousands of products are added. Optimized libraries expose this through a compute type separate from input and output types.

For a conventional FP32 matrix multiplication, call cuBLAS instead of starting with a handwritten kernel:

```
// Conceptual signature: C = alpha*A*B + beta*C
cublasSgemm(handle, opA, opB, m, n, k,
            &alpha, A, lda, B, ldb, &beta, C, ldc);
```

The important interview point is not memorizing argument order; it is knowing that a tuned library selects architecture-specific kernels and is usually faster and safer than a naive implementation.

5. Diagrams / Mental Models

```
32 bits
+-----+-----+-----+
| sign | exponent | fraction |
|  1   |    8    |   23    |
+-----+-----+-----+
           range      resolution

input products ----> partial sums ----> FP32 accumulator ----> rounded output
```

Property	FP32 consequence
More exponent bits	Large dynamic range, about 1e-38 to 3.4e38 for normals
24-bit normal significand	Roughly 7 decimal significant digits
4 bytes/value	Twice FP16/BF16 memory traffic
IEEE special values	NaN, infinities, signed zeros, subnormals

6. Common Interview Questions

- What is FP32?** It is IEEE 754 binary32: 1 sign, 8 exponent, and 23 fraction bits. Expected: mention the implicit leading bit and about seven decimal digits. Mistake: saying it has 23 total precision bits.
- Why can **0.1f** + **0.2f** differ from **0.3f**?** Most decimal fractions have repeating binary expansions and must be rounded. Expected: representation error. Mistake: blaming only the compiler.
- What is machine epsilon?** Near one, it is the gap between **1** and the next representable FP32 value. Expected: it is not a universal absolute-error bound. Mistake: applying it unchanged at every magnitude.
- Why is floating-point addition non-associative?** Each grouping rounds intermediate values differently. Expected: connect this to parallel reductions. Mistake: claiming the mathematical operation itself changed.
- What are subnormals?** Tiny values below the normal range represented with reduced precision to provide gradual underflow. Expected: mention possible flush-to-zero modes. Mistake: calling them NaNs.
- Why accumulate low-precision products in FP32?** A wider accumulator retains more partial-sum information and has a larger range. Expected: multiplication input precision is still limited. Mistake: claiming FP32 accumulation recovers information already lost during input conversion.

7. **What does FMA change?** It evaluates multiply-add with one rounding instead of separate multiply and add roundings. Expected: often faster and more accurate, but not bitwise identical. Mistake: describing it as exact overall.
8. **When should results be compared with a tolerance?** When rounding or reordering is possible. Expected: use a combined absolute and relative tolerance. Mistake: always using exact `==` or only a fixed epsilon.
9. **Why might a GPU reduction differ between runs?** Atomic update order or scheduling may change the rounding order. Expected: distinguish numerical nondeterminism from a data race. Mistake: assuming any difference proves memory corruption.
10. **When should you avoid FP32?** When accuracy/range requires FP64, or when validated lower precision yields important speed/memory gains. Expected: decide from error budget and hardware. Mistake: choosing only from peak FLOPS.

7. Deep-Dive Questions

1. **How does error grow in a dot product?** Each rounded product and addition contributes error; naive sequential accumulation has a bound that grows with the number and conditioning of terms. Pairwise summation shortens the rounding-depth from linear to logarithmic.
2. **What is catastrophic cancellation?** Subtracting nearly equal numbers removes leading significant bits, magnifying prior relative error. Algebraic reformulation or higher precision may be required.
3. **Why can `-use_fast_math` change results?** It enables faster approximations and behaviors such as flush-to-zero for some operations. Use it only after accuracy validation.
4. **Does FP32 input guarantee FP32 multiplication on modern Tensor Core paths?** No. Libraries may use TF32 Tensor Core math while exposing FP32 storage and FP32 accumulation, depending on APIs and math settings.
5. **How would you validate a GPU numerical kernel?** Compare against a higher-precision reference over random and adversarial inputs; report absolute and relative errors; explicitly test zeros, extremes, NaN/Inf policy, and cancellation-heavy cases.

8. Comparison Tables

Aspect	FP32 CUDA-core path	TF32 Tensor Core path	FP64
Storage	32-bit	FP32 at API/memory boundary	64-bit
Input significand	24 bits	About 11 bits for multiply inputs	53 bits
Typical accumulation	FP32	FP32	FP64
Main goal	General balance	Faster matrix math	Numerical accuracy/range
Typical use	General kernels	DL GEMM/convolution	Sensitive science/engineering

9. Common Mistakes

- Treating FP32 as exact real arithmetic
- Using a single absolute epsilon for values of every magnitude
- Assuming parallel and serial reductions must be bitwise equal
- Believing FP32 accumulation makes FP16/TF32 inputs fully FP32-accurate
- Ignoring overflow, underflow, NaN, and infinity during validation
- Writing a naive GEMM instead of first measuring cuBLAS

10. Edge Cases / Special Cases

- `+0.0f == -0.0f`, but their signs can affect operations such as reciprocals.
- NaN is unequal to itself.
- Overflow normally produces infinity; underflow may produce a subnormal or zero.
- Casting integers above 2^{24} to FP32 can lose unit precision.
- Atomic FP32 addition is mathematically safe from lost updates but its result can be order-dependent.
- Compiler contraction into FMA can change the last bits relative to separately rounded operations.

11. How to Explain in Interview

"FP32 is IEEE binary32 with an 8-bit exponent and 24 bits of normal-number precision including the hidden bit. It is the standard GPU accuracy baseline, but it is rounded and non-associative. I often use FP32 for accumulators and sensitive reductions even when Tensor Core inputs use FP16, BF16, TF32, or INT8."

12. Quick Revision Notes

- `float` is normally FP32 in CUDA.
- Layout: `1/8/23`; effective normal precision: 24 bits.
- Roughly seven decimal significant digits.
- Range and precision are different properties.
- FMA rounds once; reductions depend on order.
- Trap: FP32 storage can still use TF32 internally in optimized matrix libraries.

13. Practice Tasks

1. Print `16777216.0f + 1.0f` and explain the result.
2. Sum one million mixed-magnitude values sequentially, pairwise, and in FP64; compare errors.
3. Implement relative-plus-absolute approximate equality.
4. Benchmark a naive FP32 GEMM against cuBLAS SGEMM.
5. Compile a multiply-add with and without contraction and inspect the generated instructions.

14. Final Cheat Sheet

Item	Recall
Core definition	32-bit IEEE float, <code>1/8/23</code> bit fields
Why it matters	General accuracy baseline and common accumulator
Most asked	Non-associativity, epsilon, subnormals, FMA, mixed accumulation

Item	Recall
Key comparison	FP32 has more precision than TF32/FP16/BF16 but less throughput on Tensor Core workloads
One-line answer	"FP32 balances range, precision, and cost; it is approximate arithmetic, not real arithmetic."

FP16

1. Overview

Definition

FP16, IEEE 754 binary16 or half precision, uses one sign bit, five exponent bits, and ten fraction bits. Normal values have 11 bits of significand precision, roughly three to four decimal significant digits. Its maximum finite magnitude is 65504, and its narrow exponent range makes overflow and underflow important practical concerns.

Why it matters and where it is used

FP16 halves storage and memory traffic relative to FP32 and is supported at high throughput by GPU vector units and Tensor Cores. It is common for neural-network weights, activations, gradients, GEMMs, convolutions, and bandwidth-bound transforms. CUDA exposes it through `__half/__half2` in `<cuda_fp16.h>`.

Why interviewers ask

FP16 reveals whether a candidate can separate storage precision, multiplication precision, accumulation precision, and algorithm-level numerical stability.

2. Core Idea

Intuition and analogy

FP16 is a small notebook: it is cheap to carry and fast to move, but it has fewer pages for both very large/small magnitudes and fine detail. FP32 is often used as the desk where many FP16 results are combined safely.

Small example and steps

Suppose a dot product contains 1000 FP16 pairs:

1. Load 2-byte elements, reducing bandwidth.
2. Tensor Cores multiply tiles of FP16 values.
3. Accumulate the products into FP32 registers.
4. Optionally convert the final output back to FP16.

If a gradient is $1e-8$, converting it directly to FP16 may make it zero. Multiplying the loss by a scale before backpropagation shifts gradients into FP16's representable range; gradients are unscaled before the optimizer update.

3. Important Subtopics

Range versus precision

FP16 has more fraction bits than BF16 but far fewer exponent bits. It represents values near one more finely, yet overflows much sooner. Interview angle: FP16 and BF16 are both 16-bit but are not interchangeable.

FP16 storage versus FP32 accumulation

Tensor Core operations commonly multiply FP16 inputs and accumulate into FP32. This helps long sums, but input quantization and multiplication still begin from FP16 values.

`__half2`

`__half2` packs two FP16 values and allows paired SIMD-style operations. It matters for element-wise kernels, but alignment and even element counts must be handled. It is distinct from warp-wide Tensor Core matrix operations.

Loss scaling

Loss scaling addresses FP16 gradient underflow. Static scaling uses a chosen constant; dynamic scaling grows or shrinks the factor based on detected overflow. It does not add information—it uses available exponent range more effectively.

Master weights

Tiny updates may disappear if applied directly to FP16 weights. Training can keep FP32 master weights, compute fast low-precision forward/backward passes, then update the FP32 copy.

4. Real-World Example

In mixed-precision image-model training, convolutions and GEMMs use FP16 operands with FP32 accumulation. Softmax, normalization statistics, loss calculation, and optimizer state commonly remain FP32. Automatic mixed precision chooses casts and maintains a gradient scaler. The result can reduce activation memory and increase Tensor Core utilization while retaining accuracy close to an FP32 baseline.

5. Diagrams / Mental Models

```

FP32 model/input
  |
  v cast
FP16 weights + FP16 activations
  | Tensor Core multiply
  v
FP32 accumulation ----> FP32-sensitive operation
  |
  v optional cast
FP16 output

```

```
loss * scale -> backward -> unscale gradients -> overflow check -> optimizer step
```

FP16 boundary	Risk	Usual mitigation
Large activation	Overflow to Inf	Rescale, normalize, or use BF16/FP32
Tiny gradient	Underflow to zero	Loss scaling
Long reduction	Rounding accumulation	FP32 accumulator
Small weight update	Update vanishes	FP32 master weights

6. Common Interview Questions

1. **What is FP16's layout?** 1/5/10, with 11 effective significand bits for normals. Expected: narrow range and about 3–4 decimal digits. Mistake: confusing it with BF16.
2. **Why can FP16 be faster?** It reduces bytes moved and enables higher-throughput packed or Tensor Core arithmetic. Expected: speed depends on the bottleneck and supported kernel. Mistake: promising a universal 2x speedup.
3. **Why use FP32 accumulation?** To reduce rounding and overflow risk in a sum of many products. Expected: distinguish accumulator precision from input precision. Mistake: claiming the whole operation becomes FP32-equivalent.
4. **What is loss scaling?** Multiply the loss before backpropagation so small gradients do not underflow, then unscale before gradient processing/update. Mistake: saying it changes the mathematical update.
5. **Static versus dynamic loss scaling?** Static uses a fixed factor; dynamic adapts when gradients overflow. Expected: dynamic scaling trades occasional skipped steps/checking for less manual tuning.
6. **Why keep FP32 master weights?** Small updates that vanish in FP16 can still accumulate in FP32. Mistake: saying inference also always requires master weights.
7. **What is __half2?** A packed pair of half values for two-lane operations. Expected: useful for element-wise arithmetic, not the WMMA API. Mistake: calling it a Tensor Core tile.
8. **When may FP16 hurt accuracy?** Narrow dynamic range, coarse rounding, unstable reductions, exponentials, and poorly scaled data. Expected: validate end-to-end metrics. Mistake: judging only one layer's error.
9. **What values overflow FP16?** Finite magnitudes above 65504 round to infinity or the maximum finite value depending on operation/mode. Expected: know the practical maximum. Mistake: quoting FP32's range.
10. **FP16 or BF16 for training?** FP16 offers finer precision near one; BF16 offers FP32-like range and often avoids loss scaling. The best choice depends on hardware and convergence. Mistake: declaring one universally superior.

7. Deep-Dive Questions

1. **Can FP16 multiplication overflow before FP32 accumulation?** Hardware semantics and instruction variants matter, but converting or producing FP16 intermediate values can overflow even if an FP32 final result would fit. Fused Tensor Core paths are preferable to manually materializing low-precision intermediates.

2. **Why unscale before gradient clipping?** Clipping thresholds are defined for true gradient magnitudes. Clipping scaled gradients changes the algorithm.
3. **How do shape and alignment affect Tensor Core use?** Libraries tile matrices internally; dimensions, strides, alignment, and layouts influence whether efficient kernels and vectorized accesses are available. Padding can help, but profile rather than memorizing one architecture's rules.
4. **Why might an FP16 kernel be slower than FP32?** Conversion overhead, small workloads, poor layout, non-Tensor-Core operations, memory latency, or fallback kernels can dominate.
5. **How do you debug NaNs in FP16 training?** Check inputs and per-layer outputs, gradient scaler state, unscale before clipping, reduce the scale, keep sensitive operations in FP32, and compare against an FP32 reference.

8. Comparison Tables

Aspect	FP16	BF16	FP32
Bits <i>sign/exponent/fraction</i>	1/5/10	1/8/7	1/8/23
Normal precision	11 bits	8 bits	24 bits
Max finite	65504	About 3.39e38	About 3.40e38
Main strength	Better 16-bit precision	Better 16-bit range	General accuracy
Typical training issue	Underflow/overflow	Coarser rounding	Memory/throughput cost

9. Common Mistakes

- Assuming “half the bits” means exactly twice the speed
- Performing every operation, including reductions and softmax, in FP16
- Forgetting to unscale gradients before clipping or updating
- Comparing FP16 results to FP32 with exact equality
- Confusing `__half2` vector arithmetic with Tensor Core operations
- Assuming FP32 accumulation repairs FP16 input rounding

10. Edge Cases / Special Cases

- FP16's minimum positive normal is about 6.10e-5; subnormals extend lower with reduced precision.
- Casts can create zero, infinity, or duplicate adjacent source values.
- Odd-length arrays need a scalar tail when using `__half2`.
- Tensor Core eligibility depends on GPU architecture, library version, data layout, and problem shape.
- Repeatedly casting FP32↔FP16 can add extra rounding and erase performance gains.

11. How to Explain in Interview

“FP16 is IEEE half precision with a 5-bit exponent and 10 stored fraction bits. It cuts storage and bandwidth and enables fast Tensor Core math, but its range is narrow. I normally use FP16 inputs with FP32 accumulation, loss scaling for training gradients, and FP32 for sensitive reductions or master weights.”

12. Quick Revision Notes

- Layout: 1/5/10; max finite: 65504.
- Two bytes per value; `__half` and `__half2` use `<cuda_fp16.h>`.
- FP16×FP16 with FP32 accumulation is common.
- Loss scaling addresses gradient underflow.
- FP32 master weights preserve small updates.
- Trap: lower storage precision does not guarantee Tensor Core execution.

13. Practice Tasks

1. Convert logarithmically spaced FP32 values to FP16 and identify zero/Inf boundaries.
2. Implement a `__half2` vector-add kernel with an odd-length tail.
3. Compare an FP16-input dot product accumulated in FP16 and FP32.
4. Use Nsight Compute to verify whether a library GEMM issued Tensor Core operations.
5. Train a small model with static and dynamic loss scaling and log skipped updates.

14. Final Cheat Sheet

Item	Recall
Core definition	IEEE binary16, 1/5/10
Why it matters	Lower bandwidth/storage and high Tensor Core throughput
Most asked	Range, FP32 accumulation, loss scaling, master weights
Key comparison	FP16 has finer significand than BF16 but much smaller range
One-line answer	"Use FP16 where throughput matters, but protect range and accumulation with scaling and FP32."

BF16

1. Overview

Definition

BF16 (bfloat16) is a 16-bit floating-point format with one sign bit, eight exponent bits, and seven fraction bits. It preserves FP32's exponent width and therefore approximately its dynamic range, but keeps only eight bits of normal-number significand precision.

Why it matters and where it is used

BF16 provides two-byte storage and Tensor Core throughput while greatly reducing the FP16 overflow/underflow problem. It is widely used for deep-learning training, LLM workloads, GEMMs, convolutions, activations, and communication buffers. CUDA exposes `__nv_bfloat16` and `__nv_bfloat162` through `<cuda_bf16.h>` on supported devices.

Why interviewers ask

It tests whether the candidate understands that bit budget can be spent on **range** or **precision**, and why training may prefer a coarser format with safer range.

2. Core Idea

Intuition and analogy

Imagine FP32 scientific notation written with the same exponent but fewer significant digits. BF16 keeps the “size of number” field and shortens the “detail” field. It can talk about roughly the same huge and tiny scales as FP32, but describes each scale less precisely.

Small example and steps

Near 1.0 , BF16’s spacing is $2^{-7} = 0.0078125$, much larger than FP16’s 2^{-10} . But 100000 overflows FP16 and remains finite in BF16.

1. An FP32 tensor is rounded to BF16; low fraction bits are discarded with the chosen rounding rule.
2. BF16 values are loaded at two bytes each.
3. Tensor Cores multiply BF16 tiles.
4. Partial sums commonly accumulate in FP32.
5. Sensitive state stays FP32; outputs may be stored as BF16.

3. Important Subtopics

FP32-like exponent range

BF16 uses the same exponent-field width as FP32. This makes it much harder than FP16 to overflow activations or underflow gradients solely because of exponent range. It does not mean BF16 has FP32 accuracy.

Coarse significand

With only seven fraction bits, adjacent values near one differ by about 0.0078125 . Individual weights and updates are rounded more aggressively than FP16. FP32 accumulation and master state still matter.

Conversion and rounding

Simply chopping the lower 16 bits of FP32 is truncation and can bias results. Correct conversions commonly use round-to-nearest-even. Interviewers may ask why a cast is more than taking the upper bits.

BF16 Tensor Core path

On supported NVIDIA architectures, BF16 matrix operands can feed Tensor Cores with FP32 accumulators. An optimized library should be the default interface; it can choose tiling and fallbacks.

Training stability

BF16 often does not require FP16-style loss scaling because of its exponent range, but “often” is not “never.” Algorithms can still generate Inf/NaN or lose tiny updates through coarse precision.

4. Real-World Example

An LLM training job stores activations and many matrix operands in BF16. Attention score reductions, softmax normalization, optimizer states, and selected master weights remain FP32. Compared with FP32, the job moves about half the bytes for BF16 tensors and uses Tensor Cores. Compared with FP16, it is less likely to overflow when activation magnitudes vary across layers.

5. Diagrams / Mental Models

```
FP32: |S| EEEEEEEE | FFFFFFFFFFFFFFFFFFFFFFFF |
BF16: |S| EEEEEEEE | FFFFFFFF                |
      same range    fewer significant bits
```

BF16 A x BF16 B --> FP32 accumulator --> BF16 or FP32 output

Choice	What is preserved	What is sacrificed
FP16	More 16-bit significand detail	Exponent range
BF16	FP32-like exponent range	Significand detail
FP32	Both wider range and detail	Memory and throughput

6. Common Interview Questions

- What is BF16's bit layout?** 1/8/7. Expected: same exponent width as FP32, eight effective normal significand bits. Mistake: saying it is IEEE FP16.
- Why is BF16 useful for training?** It keeps FP32-like range while halving storage and enabling Tensor Cores. Mistake: claiming it has FP32 precision.
- BF16 versus FP16?** BF16 has much wider range; FP16 has three more stored fraction bits. Expected: range-versus-precision trade-off. Mistake: comparing only total bits.
- Does BF16 need loss scaling?** Usually less often than FP16, because it preserves exponent range. Expected: validation is still required. Mistake: saying numerical failure is impossible.
- Why accumulate BF16 in FP32?** To reduce error when summing many coarse products and provide wider accumulator state. Mistake: claiming exact equivalence to FP32-input GEMM.
- How is FP32 converted to BF16?** By rounding to a representable BF16 value, commonly nearest-even. Mistake: assuming blind truncation is always correct.
- What CUDA type represents BF16?** `__nv_bfloat16` from `<cuda_bfloat16.h>`; paired operations use `__nv_bfloat162`. Mistake: using `__half` and assuming identical bits.
- Can BF16 exactly represent all small integers?** Only up to its significand limit; normal precision is eight bits, so unit spacing is lost much earlier than in FP16/FP32. Mistake: inferring integer exactness from exponent range.
- Why can BF16 be slower than expected?** Unsupported hardware, casts, small shapes, fallback kernels, bandwidth outside BF16 tensors, or non-matrix work. Mistake: equating the data type with guaranteed Tensor Core execution.
- When prefer FP16?** When values are safely scaled and its finer significand gives better accuracy, or when the target hardware/library supports FP16 better. Mistake: choosing solely by maximum range.

7. Deep-Dive Questions

1. **Why does BF16 retain roughly FP32's range despite half the storage?** Both allocate eight exponent bits with a similar bias; BF16 removes fraction bits rather than exponent bits.
2. **Can FP32 master weights still matter in BF16 training?** Yes. An update smaller than the BF16 spacing around a weight can disappear when stored directly in BF16.
3. **What error does BF16 introduce in GEMM?** Inputs are rounded to coarse BF16 values, their products reflect that error, and FP32 accumulation limits additional summation error but cannot restore discarded input bits.
4. **How would you choose BF16 versus TF32?** BF16 reduces memory footprint; TF32 generally keeps FP32 storage/API data but reduces multiply precision on Tensor Cores. Consider bandwidth, conversions, accuracy, and library support.
5. **What should a validation suite contain?** End-to-end metric checks, layerwise comparisons to FP32, magnitude extremes, cancellation, long reductions, NaN/Inf monitoring, and representative production distributions.

8. Comparison Tables

Feature	BF16	FP16	TF32
Stored tensor size	16 bits	16 bits	Usually FP32 storage
Exponent bits	8	5	8
Fraction bits used	7	10	10
Range	About FP32	Narrow	About FP32
Typical use	Training operands/storage	Training/inference operands	Accelerated FP32 GEMM/convolution

9. Common Mistakes

- Saying BF16 is just another name for FP16
- Equating FP32-like range with FP32-like accuracy
- Truncating FP32 without considering rounding bias
- Assuming loss scaling can never help
- Keeping sensitive optimizer state and reductions in BF16 without validation
- Assuming every GPU supports native BF16 Tensor Core operations

10. Edge Cases / Special Cases

- BF16's coarse spacing can erase small residual updates even when no underflow occurs.
- Different conversion paths or rounding modes may produce different last BF16 bits.
- NaN payload details need not survive conversion identically.
- Hardware and compute-capability support must be checked at deployment.
- A BF16 workload can remain bandwidth-bound or dominated by FP32 operations.

11. How to Explain in Interview

"BF16 is a 16-bit float with FP32's eight exponent bits but only seven stored fraction bits. It trades precision for range, making it attractive for neural-network training because it avoids much of FP16's overflow and underflow risk. I still use FP32 accumulation and keep sensitive state in FP32."

12. Quick Revision Notes

- Layout: 1/8/7.
- Range similar to FP32; precision lower than FP16.
- CUDA type: `__nv_bfloat16`.
- Common path: BF16 multiply, FP32 accumulate.
- Loss scaling is less commonly necessary than with FP16.
- Trap: exponent range and significand precision are independent.

13. Practice Tasks

1. Convert values near 1.0 to BF16 and FP16; list representable spacing.
2. Test 100000, 1e-20, and small weight updates in FP16 and BF16.
3. Compare BF16 and FP16 GEMM error against an FP64 reference.
4. Inspect profiler metrics to confirm a BF16 Tensor Core kernel.
5. Keep a training model's matmuls in BF16 but force softmax/reductions to FP32; compare stability.

14. Final Cheat Sheet

Item	Recall
Core definition	16-bit float, 1/8/7
Why it matters	Two-byte data with FP32-like dynamic range
Most asked	BF16 vs FP16, accumulation, rounding, loss scaling
Key comparison	BF16 spends bits on range; FP16 spends more on precision
One-line answer	"BF16 keeps FP32's exponent range in 16 bits, then relies on FP32 accumulation for robust sums."

TF32

1. Overview

Definition

TensorFloat-32 (TF32) is an NVIDIA Tensor Core computation format designed to accelerate matrix operations whose data is supplied as FP32. It uses FP32's eight exponent bits, about FP16's significand precision (10 stored fraction bits plus the normal leading bit), and commonly accumulates into FP32.

TF32 is not normally used as a compact 19-bit array type. Tensors stay FP32 in memory; eligible matrix-operation inputs are rounded internally to TF32 precision.

Why it matters and where it is used

TF32 lets existing FP32 deep-learning GEMMs and convolutions use Tensor Cores with few or no model-code changes. It is used through optimized libraries and frameworks on supported NVIDIA GPUs. It matters when FP32 range and interfaces are convenient but full FP32 multiply precision is unnecessary.

Why interviewers ask

TF32 catches a common misconception: API type, storage type, multiply precision, and accumulator precision can all differ.

2. Core Idea

Intuition and analogy

Imagine submitting a detailed FP32 measurement to a fast calculator. The calculator keeps the measurement's scale but rounds its fine digits before multiplication, then writes the running sum in the larger FP32 notebook. Memory remains FP32-sized, so TF32 mainly accelerates arithmetic rather than halving storage traffic.

Small example and steps

For an FP32 GEMM on a TF32-enabled library path:

1. **A** and **B** are stored and passed as FP32.
2. Tensor Core input hardware rounds their significands to TF32 precision.
3. Tiles are multiplied at Tensor Core throughput.
4. Products accumulate in FP32.
5. **C** is returned as FP32.

Two FP32 values that differ only in low fraction bits may become the same TF32 input. That is the speed/accuracy trade-off.

3. Important Subtopics

Storage versus compute format

TF32 does not provide FP16-like memory savings because input/output tensors are normally FP32. This distinction is a frequent interview test.

Range and precision

The exponent range is approximately FP32's; the multiply input precision is around 11 significand bits. FP32 accumulation limits summation error but cannot reconstruct rounded input bits.

Library control

cuBLAS, cuBLASLt, cuDNN, and higher-level frameworks expose controls that influence whether TF32 is permitted. Names and defaults can change by version, so production code should set the required numerical

policy explicitly and benchmark the actual path.

Eligibility and fallback

Only suitable operations and supported devices use Tensor Cores. Small, irregular, unsupported, or policy-disabled operations may use conventional FP32 instructions. A `float` API does not prove either path.

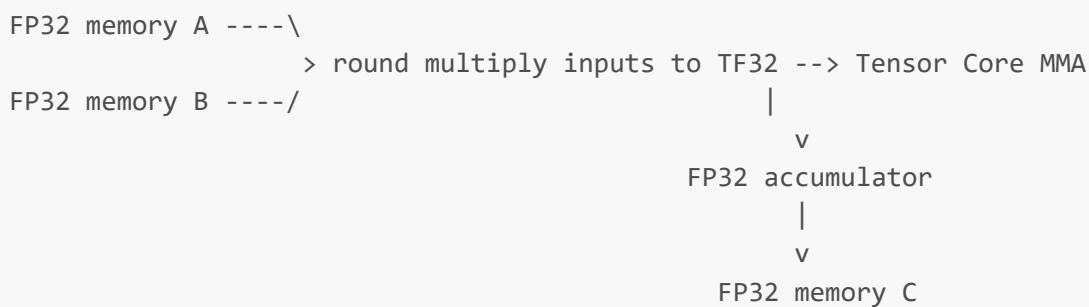
Validation

Compare task-level accuracy and numerical error against a strict FP32 or FP64 reference. Ill-conditioned matrix problems may magnify the reduced input precision even when ordinary deep-learning workloads tolerate it.

4. Real-World Example

A vision model originally trained with FP32 tensors runs its convolution and linear layers through cuDNN/cuBLAS. Enabling the framework's TF32 policy allows eligible operations to use Tensor Cores. Model parameters still occupy four bytes each, while matrix arithmetic becomes faster. Layer normalization, activation functions, and other non-matrix work do not automatically gain TF32 Tensor Core speed.

5. Diagrams / Mental Models



Question	TF32 answer
Are tensors stored in 19 bits?	No, normally FP32 storage
Is exponent range reduced to FP16?	No, it is FP32-like
Is multiply precision full FP32?	No, low fraction bits are rounded away
Is accumulation commonly FP32?	Yes
Does every <code>float</code> operation use TF32?	No, eligible Tensor Core matrix operations do

6. Common Interview Questions

- What is TF32?** A Tensor Core computation format with FP32-like exponent range and reduced significand precision. Expected: FP32 accumulation and FP32 storage boundary. Mistake: calling it a standard C++ 19-bit type.
- Does TF32 reduce model memory?** Generally no, because tensors remain FP32 in memory. Mistake: promising FP16-like memory savings.

3. **How does TF32 differ from FP32?** Its matrix multiply inputs use fewer significand bits; FP32 uses 24 normal significand bits. Mistake: saying only the exponent differs.
4. **How does TF32 differ from FP16?** TF32 normally uses FP32 storage and FP32-like range; FP16 is a true 16-bit storage format with a 5-bit exponent. Mistake: treating equal significand precision as equal formats.
5. **Why can TF32 accelerate unchanged FP32 code?** Optimized libraries can route eligible FP32 GEMM/convolution calls to Tensor Cores. Mistake: saying all scalar FP32 arithmetic changes.
6. **Does FP32 accumulation make TF32 exact?** No. Rounded-away input bits are already lost. Mistake: considering only the accumulator.
7. **When should TF32 be disabled?** When validation shows unacceptable error, strict reproducibility is required, or an algorithm is sensitive/ill-conditioned. Mistake: disabling it solely because results are not bitwise identical.
8. **How do you confirm TF32 is used?** Inspect library logs or GPU profiler instruction/kernel metrics and compare configured math policy. Mistake: inferring it from data type alone.
9. **Can ordinary element-wise float operations use TF32?** TF32 targets Tensor Core matrix operations, not general scalar FP32 arithmetic. Mistake: assuming a global change to float semantics.
10. **Why might TF32 show little speedup?** The workload may be bandwidth-bound, small, poorly shaped, dominated by non-GEMM work, or already bottlenecked elsewhere. Mistake: comparing only advertised peak throughput.

7. Deep-Dive Questions

1. **What happens to low FP32 mantissa bits?** Inputs are rounded to the precision accepted by the TF32 Tensor Core operation; exact implementation details are architecture/API-defined, so use documented numerical guarantees rather than bit-hacking a storage assumption.
2. **Can iterative refinement recover accuracy?** Sometimes. A low-precision solve can produce a candidate while higher-precision residual computation and correction recover accuracy, provided conditioning and convergence permit it.
3. **Why is conditioning important?** A problem with a large condition number amplifies small input and rounding perturbations; TF32's reduced input precision can then create large output error.
4. **How is explicit WMMA TF32 different from a library call?** WMMA may require explicit conversion such as `__float_to_tf32` and fixed fragments, while a library selects kernels, conversions, layouts, and fallbacks.
5. **How would you make numerical policy reproducible across deployments?** Pin/test software versions, explicitly configure allowed compute modes, record GPU architecture, use deterministic algorithm settings where available, and validate tolerances rather than relying on defaults.

8. Comparison Tables

Aspect	TF32	FP32	BF16
Usual storage	FP32	FP32	16-bit
Exponent bits used	8	8	8
Normal significand precision	About 11 bits	24 bits	8 bits
Accumulator	Commonly FP32	FP32	Commonly FP32

Aspect	TF32	FP32	BF16
Main benefit	Faster FP32-interface matrix math	Accuracy baseline	Memory + compute savings

9. Common Mistakes

- Calling TF32 a 32-bit IEEE storage format
- Expecting memory footprint to halve
- Believing every FP32 instruction becomes TF32
- Ignoring library policy and fallback behavior
- Treating FP32 accumulation as full FP32 multiplication
- Using bitwise equality as the only correctness test

10. Edge Cases / Special Cases

- A supported GPU is necessary but not sufficient; the library, operation, shape, and policy also matter.
- TF32 can be unsuitable for sensitive linear algebra even when it works well for neural networks.
- Framework defaults may change; set the desired behavior explicitly.
- Results may differ from strict FP32 in low bits and sometimes more for poorly conditioned data.
- Explicit WMMA TF32 uses `float`-typed values that have been rounded for TF32 consumption.

11. How to Explain in Interview

"TF32 is a Tensor Core compute mode for FP32 matrix workloads. Inputs stay FP32 in memory and retain FP32-like range, but multiplication uses about 11 significand bits and accumulation is normally FP32. It accelerates GEMM and convolution without FP16-style memory savings, so I enable it only after accuracy validation."

12. Quick Revision Notes

- FP32 storage; reduced-precision Tensor Core multiply.
- Eight exponent bits, about ten stored fraction bits used.
- FP32 accumulation is common.
- Targets GEMM/convolution, not arbitrary scalar operations.
- Validate ill-conditioned problems carefully.
- Trap: `float` inputs do not imply strict FP32 multiply precision.

13. Practice Tasks

1. Run the same cuBLAS FP32 GEMM with TF32 allowed and disallowed; measure speed and error.
2. Compare well-conditioned random matrices with a Hilbert-like ill-conditioned case.
3. Profile a model and identify which layers receive Tensor Core speedup.
4. Measure memory use for TF32 versus FP16 tensors and explain the difference.
5. Implement one WMMA TF32 tile conversion experiment on supported hardware.

14. Final Cheat Sheet

Item	Recall
Core definition	FP32-range, reduced-significant Tensor Core compute format
Why it matters	Accelerates eligible FP32 matrix workloads
Most asked	Storage vs compute, FP32 accumulation, accuracy, enable/disable policy
Key comparison	TF32 saves compute time; BF16/FP16 also save storage/bandwidth
One-line answer	"TF32 keeps the FP32 interface and range, rounds matrix inputs, and accumulates on Tensor Cores in FP32."

INT8

1. Overview

Definition

INT8 is an 8-bit integer representation. For signed two's-complement data its range is **-128** to **127**; unsigned INT8 ranges from **0** to **255**. In machine learning, real-valued tensors are mapped to integers using a scale and sometimes a zero point:

$$q = \text{clamp}(\text{round}(x/s) + z), \quad x \approx s(q - z)$$

NVIDIA inference paths often use symmetric signed quantization where **z = 0**, though the exact scheme depends on the framework and operator.

Why it matters and where it is used

INT8 uses one quarter of FP32's storage, reduces bandwidth, and enables high-throughput integer dot products/Tensor Core matrix operations. It is common in neural-network inference, edge deployment, vision, speech, recommendation, and quantized transformer workloads.

Why interviewers ask

INT8 tests quantization, scaling, calibration, accumulator width, saturation, accuracy trade-offs, and the difference between integer representation and real-number semantics.

2. Core Idea

Intuition and analogy

A thermometer may record only integer marks even though temperature is continuous. Choose what one mark means (the **scale**) and where real zero lands (the **zero point**). A narrow scale gives fine resolution but clips extremes; a wide scale preserves extremes but gives coarser steps.

Small example and steps

For symmetric scale **s = 0.1**:

```
real x = 1.26
q = round(1.26 / 0.1) = 13
dequantized x' = 13 * 0.1 = 1.3
```

1. Collect or learn tensor ranges.
2. Choose per-tensor, per-channel, or per-block scales.
3. Quantize values with rounding and clipping.
4. Multiply INT8 operands.
5. Accumulate many products in INT32.
6. Rescale/requantize to the next quantized tensor or dequantize to floating point.

3. Important Subtopics

Quantization error

Rounding error maps values to nearby grid points. **Clipping error** saturates values outside the representable range. Increasing the scale reduces clipping but makes each step larger. Calibration balances these errors.

Symmetric and asymmetric quantization

Symmetric quantization centers the integer grid around zero and simplifies arithmetic. Asymmetric quantization uses a zero point to represent an offset range more efficiently. Interview angle: zero-point correction adds terms to dot-product arithmetic.

Per-tensor versus per-channel scaling

One scale per tensor is simple and compact. Per-channel weight scales adapt to channels with different ranges and often improve accuracy at a small metadata/implementation cost.

PTQ and QAT

Post-training quantization (PTQ) converts a trained model using representative calibration data.

Quantization-aware training (QAT) simulates rounding/clipping during training so weights adapt; it often preserves accuracy better but costs retraining effort.

INT32 accumulation

An INT8 product fits within roughly 16 bits, but a dot product adds many products. INT32 accumulators are standard. Overflow is still possible for sufficiently large K or biased/extreme inputs; accumulator width is not infinite.

4. Real-World Example

An image-classification server receives FP32 pixels, quantizes activations using calibration-derived scales, runs convolution/GEMM layers with INT8 inputs and INT32 accumulation, then requantizes between layers. TensorRT or another optimized runtime fuses quantize/dequantize operations and chooses Tensor Core kernels. The team validates top-1 accuracy and latency on representative production images before deployment.

5. Diagrams / Mental Models

```

FP32 tensor
  | divide by scale, round, clamp
  v
INT8 tensor ----> INT8 x INT8 ----> INT32 accumulation
                        |
                        multiply by combined scale
                        v
                        INT8 next layer or FP32 output

```

Scale choice	Benefit	Cost
Small s	Fine resolution	More clipping
Large s	Covers outliers	More rounding error
Per-channel	Adapts to weight channels	More scales/constraints
Per-tensor	Simple	One outlier can waste range

6. Common Interview Questions

- What does INT8 quantization do?** It maps real values onto a finite integer grid using scale, rounding, optional zero point, and clipping. Mistake: describing it as a normal C++ cast.
- Why is INT8 fast?** Smaller data reduces storage/bandwidth and supported GPUs execute high-throughput integer dot products/Tensor Core operations. Mistake: assuming every layer accelerates.
- Why accumulate in INT32?** A sum of many INT8 products needs more range. Mistake: saying INT8 multiplication directly produces an INT8 final sum.
- What is calibration?** Running representative data to determine activation ranges/scales for PTQ. Expected: representative distribution matters. Mistake: calibrating on random noise.
- PTQ versus QAT?** PTQ is simpler and needs no retraining; QAT models quantization during training and usually handles accuracy-sensitive models better. Mistake: saying QAT removes all error.
- Symmetric versus asymmetric quantization?** Symmetric uses a zero-centered scale, often zero point zero; asymmetric shifts the grid with a zero point. Expected: arithmetic simplicity versus range use.
- Per-tensor versus per-channel?** One scale for all values versus separate scales along an axis, often output channels for weights. Mistake: assuming activations and weights always use the same scheme.
- What causes quantization error?** Rounding/discretization and clipping/saturation. Mistake: mentioning only fewer bits.
- Can INT8 represent fractional real values?** The stored value is integer, but scale gives it a fractional real interpretation. Mistake: saying no fractional quantity can be modeled.
- When should INT8 not be used?** When validated accuracy loss is unacceptable, operators lack support, conversion overhead dominates, or dynamic ranges are difficult to quantize. Mistake: forcing every layer to INT8.

7. Deep-Dive Questions

1. **Derive a quantized dot product with zero points.** For $x \approx s_x(qx - zx)$ and $w \approx s_w(qw - zw)$, the dot product scales $\text{sum}((qx - zx)(qw - zw))$ by $s_x * s_w$; expansion introduces correction sums when zero points are nonzero.
2. **Can INT32 accumulation overflow?** Yes. A conservative bound is $K * \max_abs(a) * \max_abs(b)$. Large K or extreme values can approach $2^{31}-1$; kernels may split/reduce or impose constraints.
3. **Why do outliers hurt quantization?** A per-tensor scale large enough for rare extremes makes the grid too coarse for most values. Clipping, per-channel/block scaling, or model transformation can help.
4. **Why fuse requantization?** Writing INT32 results and launching a separate conversion kernel costs bandwidth and latency; an epilogue can apply scale, bias, activation, and output conversion before storing.
5. **How do you evaluate an INT8 deployment?** Measure task accuracy, per-layer error, saturation rates, representative percentile distributions, latency including Q/DQ, throughput, memory, and unsupported fallbacks.

8. Comparison Tables

Aspect	INT8	FP16	FP32
Storage	1 byte	2 bytes	4 bytes
Representation	Uniform integer grid after scaling	Floating point	Floating point
Typical accumulator	INT32	FP32	FP32
Main workflow	Quantization/calibration or QAT	Casting/mixed precision	Baseline
Typical role	Inference	Training and inference	Training/reference/general compute
PTQ	QAT		
No retraining required	Training/fine-tuning required		
Needs representative calibration data	Simulates quantization during training		
Faster adoption	Usually better difficult-model accuracy		
Best first attempt	Use when PTQ misses accuracy target		

9. Common Mistakes

- Using min/max from unrepresentative calibration data
- Forgetting clipping when describing quantization
- Assuming INT32 accumulation cannot overflow
- Applying one scale to every channel despite severe range imbalance
- Measuring kernel time but excluding quantize/dequantize overhead
- Assuming smaller model size guarantees lower end-to-end latency

10. Edge Cases / Special Cases

- Signed INT8 is asymmetric by one value (-128 versus 127). Some symmetric schemes deliberately use only $[-127, 127]$.
- Rounding ties policy can affect bias and reproducibility.
- ReLU-like nonnegative activations may benefit from asymmetric or unsigned representations if supported.
- Calibration caches may depend on software version, fusion decisions, and target configuration.
- Softmax, normalization, or numerically sensitive layers may remain in floating point.
- Dynamic activation quantization adds runtime scale computation but handles changing distributions.

11. How to Explain in Interview

"INT8 inference represents real tensors on an 8-bit integer grid using scales and sometimes zero points. Hardware multiplies INT8 values and normally accumulates in INT32, then requantizes or dequantizes. The engineering challenge is selecting scales that balance rounding and clipping, using PTQ calibration or QAT, and validating end-to-end accuracy."

12. Quick Revision Notes

- Signed range: $[-128, 127]$.
- Quantization: scale, optional zero point, rounding, clamp.
- Common math: $\text{INT8} \times \text{INT8} \rightarrow \text{INT32}$ accumulation.
- PTQ uses representative calibration; QAT trains with simulated quantization.
- Per-channel weights usually improve accuracy.
- Trap: INT8 is a representation plus scale, not merely a cast.

13. Practice Tasks

1. Quantize a small vector symmetrically and calculate rounding/clipping error by hand.
2. Compare per-tensor and per-channel quantization for a weight matrix with one outlier channel.
3. Compute a safe upper bound for an INT32 dot-product accumulator.
4. Calibrate a small model with representative and biased datasets; compare accuracy.
5. Profile end-to-end latency with and without fused Q/DQ operations.

14. Final Cheat Sheet

Item	Recall
Core definition	8-bit integer values interpreted through quantization parameters
Why it matters	Very low bandwidth/storage and high inference throughput
Most asked	Scale/zero point, PTQ/QAT, calibration, INT32 accumulation
Key comparison	INT8 needs quantization; FP16 is directly floating point
One-line answer	"INT8 trades continuous precision for a scaled integer grid, then uses wide accumulation and careful calibration."

Mixed-Precision Computing

1. Overview

Definition

Mixed-precision computing deliberately uses multiple numeric formats within one algorithm. Low precision handles operations that tolerate it and benefit from lower bandwidth or higher throughput; higher precision protects sensitive state, reductions, and outputs.

Why it matters and where it is used

It is the software strategy that turns FP16, BF16, TF32, and INT8 hardware into useful performance without blindly sacrificing correctness. It appears in deep-learning training and inference, iterative solvers, dense linear algebra, scientific simulation, image processing, and communication compression.

Why interviewers ask

The topic tests end-to-end numerical reasoning. A strong candidate does not answer “use FP16 everywhere”; they identify sensitive operations, scaling, accumulation types, validation, and library support.

2. Core Idea

Intuition and analogy

Use a rough pencil for large shapes and a fine pen for details. Most expensive matrix work can tolerate a rougher format, while small updates, totals, normalization denominators, or convergence checks need the fine pen.

Small example and steps

A training iteration may use:

1. FP32 master parameters and optimizer state.
2. BF16 or FP16 copies for forward GEMMs/convolutions.
3. FP32 accumulators inside those operations.
4. FP32 for softmax, normalization statistics, and loss.
5. Scaled low-precision backward computation when using FP16.
6. Unscaled FP32 gradients for clipping and parameter update.

The formats are chosen per operation and per tensor role, not by one global switch.

3. Important Subtopics

Storage, operand, compute, accumulator, and output types

These are separate choices. A tensor can be stored in FP32, rounded to TF32 for multiplication, accumulated in FP32, and stored back in FP32. Interviewers expect candidates to name these boundaries.

Automatic mixed precision (AMP)

AMP applies allow/deny policies for operations, inserts casts, and usually integrates gradient scaling. It reduces manual work but remains a policy engine, not a proof of accuracy.

Loss scaling and overflow detection

FP16 gradients can underflow. Scale the loss, backpropagate, unscale gradients, verify they are finite, then clip/update. A dynamic scaler lowers scale on overflow and can increase it after stable steps.

Numerically sensitive operations

Long reductions, variance, softmax exponentials and denominators, optimizer state, and some losses often require FP32. The exact list depends on implementation and input distribution.

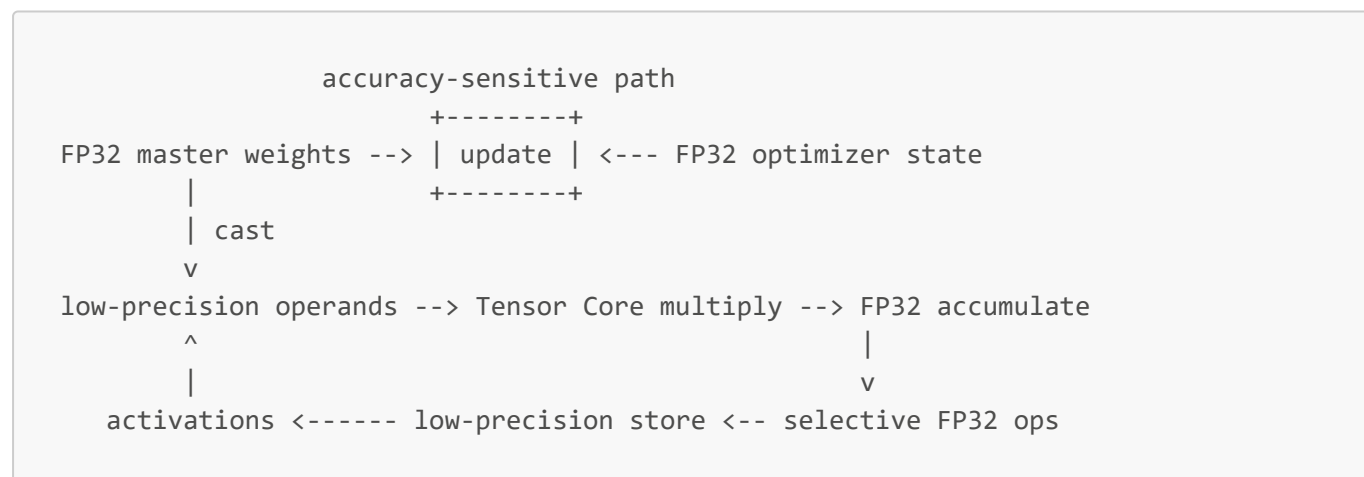
Performance model

Mixed precision helps if it reduces the actual bottleneck. Compute-bound Tensor Core GEMM, bandwidth-bound tensor movement, and memory capacity can benefit differently. Casts, synchronization, small kernels, and unsupported fallbacks can erase gains.

4. Real-World Example

In transformer training, large linear projections use BF16 inputs with FP32 accumulation. Attention softmax performs max subtraction and reductions in FP32. Optimizer moments remain FP32. Activation tensors stored in BF16 reduce memory enough to increase batch size. The framework's AMP policy manages most casts, while profiling confirms Tensor Core use and validation confirms matching loss/convergence.

5. Diagrams / Mental Models



Layer/operation	Common choice	Reason
GEMM/convolution inputs	FP16 or BF16	Throughput and memory
GEMM accumulator	FP32	Long sums
Softmax/reductions	FP32 internally	Stability
Optimizer moments	FP32	Preserve small history/update
Inference weights	FP16/BF16/INT8	Throughput and capacity

6. Common Interview Questions

1. **What is mixed precision?** Intentional use of multiple formats in one computation. Expected: give operand/accumulator example. Mistake: defining it as converting the whole program to FP16.
2. **Why is it faster?** Lower bytes and Tensor Core throughput can improve memory- and compute-bound regions. Mistake: guaranteeing speedup without profiling.
3. **Why keep FP32 master weights?** To preserve updates smaller than low-precision spacing. Mistake: confusing master weights with low-precision forward copies.
4. **What is AMP?** Framework policy that selects precision, inserts casts, and often handles scaling. Mistake: saying it automatically guarantees identical accuracy.
5. **Which operations stay FP32?** Commonly reductions, normalization statistics, softmax internals, loss, and optimizer state. Expected: workload-dependent. Mistake: memorizing a universal list.
6. **Why does FP16 need loss scaling more than BF16?** FP16 has a much narrower exponent range. Mistake: saying BF16 has more fraction bits.
7. **When does mixed precision not help?** Unsupported hardware/operators, small workloads, cast overhead, launch/communication bottlenecks, or strict accuracy constraints. Mistake: using peak FLOPS as an end-to-end prediction.
8. **How do you validate it?** Compare task metrics, convergence, per-layer errors, NaN/Inf, and performance against a trusted baseline on representative data. Mistake: checking only one output.
9. **What is an FP32 accumulator?** Register/state that sums lower-precision products in FP32. Mistake: claiming it restores lower input bits.
10. **How do optimized libraries help?** They choose supported Tensor Core kernels, tiles, layouts, fusion, and architecture-specific implementations. Mistake: hand-coding WMMA before measuring a library call.

7. Deep-Dive Questions

1. **How would you design a precision policy for a new model?** Begin with a validated FP32 baseline, enable library/AMP defaults, find accuracy or overflow failures, keep only sensitive operations/state wider, profile fallbacks, and record the policy explicitly.
2. **What is iterative refinement?** Compute a fast approximate solution at low precision, calculate residuals in higher precision, and solve corrections iteratively. It can achieve higher accuracy if the problem is sufficiently well-conditioned.
3. **How can distributed training complicate mixed precision?** Gradient communication format, reduction precision/order, scaling consistency, overflow coordination, and nondeterministic collective algorithms all affect correctness and convergence.
4. **Why can more low-precision FLOPS fail to improve throughput?** Amdahl's law: nonaccelerated layers, input pipelines, communication, memory movement, or kernel-launch overhead can dominate total time.
5. **How does fusion help?** A fused epilogue can keep accumulators in registers, apply bias/activation/scale, and write the final type once, reducing conversions, memory traffic, and launches.

8. Comparison Tables

Strategy	Inputs	Accumulation/state	Main benefit	Main risk
Strict FP32	FP32	FP32	Simple baseline	More bytes/lower matrix throughput
FP16 mixed	FP16	FP32 + master state	Precision + throughput	Narrow range, scaling needed
BF16 mixed	BF16	FP32 + master state	Range + throughput	Coarse significand
TF32	FP32 storage, TF32 compute	FP32	Minimal code/data change	Reduced multiply precision, no memory saving
INT8 inference	INT8 + scales	INT32/FP32 epilogue	Maximum inference efficiency	Calibration/accuracy complexity

9. Common Mistakes

- Treating precision as a global type rather than a per-operation policy
- Failing to unscale before clipping gradients
- Leaving extra casts between every layer
- Assuming library selection guarantees Tensor Core execution
- Ignoring convergence and checking only one inference batch
- Optimizing GEMM while data loading or communication dominates

10. Edge Cases / Special Cases

- Overflow and underflow checks must occur before a bad value contaminates optimizer state.
- Gradient accumulation across microbatches may need FP32 even when individual gradients are low precision.
- Some operations internally upcast despite low-precision inputs; this is usually intentional.
- Determinism settings can select slower kernels or change reduction order.
- Checkpoint formats and resume logic must preserve master weights, scaler state, and optimizer precision.
- In inference, quantization transitions can outweigh compute savings for tiny batches.

11. How to Explain in Interview

"Mixed precision assigns low precision to expensive tolerant operations and higher precision to sensitive accumulation and state. A common training path uses FP16 or BF16 matrix inputs, FP32 accumulators and optimizer state, plus loss scaling for FP16. I rely on optimized libraries/AMP, then validate accuracy and profile actual Tensor Core use."

12. Quick Revision Notes

- Separate storage, input, multiply, accumulator, and output precision.
- FP16: loss scaling often needed; BF16: safer range.

- Keep sensitive reductions and state in FP32.
- AMP automates policy, not validation.
- Library first; custom WMMA/MMA only for missing fusion or operation.
- Trap: theoretical Tensor Core throughput is not application speedup.

13. Practice Tasks

1. Draw a precision map for one training iteration.
2. Add AMP to a small model; log scaler changes and skipped steps.
3. Force softmax into low precision, observe error, then restore FP32 internals.
4. Profile cast kernels and remove redundant conversions.
5. Implement iterative refinement for a small linear system using a low-precision inner solve.

14. Final Cheat Sheet

Item	Recall
Core definition	Multiple numeric formats chosen by operation/tensor role
Why it matters	Captures low-precision speed without giving up critical accuracy
Most asked	Loss scaling, FP32 master state, sensitive ops, validation
Key comparison	FP16, BF16, TF32, and INT8 solve different range/storage/workflow needs
One-line answer	"Compute cheaply where error is tolerated; accumulate and update carefully where it is not."

Tensor Core Architecture

1. Overview

Definition

Tensor Cores are specialized GPU execution units optimized for small matrix multiply-accumulate operations. Rather than issuing one scalar multiply or add per instruction, cooperating threads issue a tile operation implementing a form of:

$$D_{m \times n} = A_{m \times k} B_{k \times n} + C_{m \times n}$$

The supported shapes, operand formats, accumulator formats, sparsity modes, and instruction mechanisms depend on GPU architecture.

Why it matters and where it is used

Matrix multiplication is the computational center of dense layers, convolutions transformed into GEMM-like operations, attention, scientific tensor contractions, and many signal-processing algorithms. Tensor Cores provide much higher matrix throughput than composing the same work from scalar instructions.

They are used by cuBLAS/cuBLASLt, cuDNN, CUTLASS-based kernels, TensorRT, deep-learning frameworks, and selected custom CUDA kernels.

Why interviewers ask

Interviewers want to know whether a candidate understands warp cooperation, tiling, data reuse, arithmetic intensity, precision/accumulation, and why peak hardware capability is useless without a good memory pipeline.

2. Core Idea

Intuition and analogy

A CUDA core is like one cashier handling individual items. A Tensor Core is a checkout station designed for an entire standardized tray. It is very fast when data arrives in the right tray shape and layout. If workers spend all their time fetching scattered items or repacking trays, the station sits idle.

Small example and step-by-step execution

To compute a large GEMM, a high-performance kernel generally:

1. Partitions output **C** into thread-block tiles.
2. Loads tiles of **A** and **B** from global memory, often into shared memory.
3. Arranges/pads data to support coalesced accesses and avoid shared-memory bank conflicts.
4. Lets warps or warp groups load register fragments.
5. Issues many matrix multiply-accumulate operations over the **K** dimension.
6. Keeps accumulator fragments in registers.
7. Applies an epilogue such as scaling, bias, activation, or quantization.
8. Stores each output tile coalescently.

The hardware instruction is only step 5. Most kernel engineering is feeding it efficiently.

3. Important Subtopics

Warp-level cooperation

Classic Tensor Core MMA operations distribute matrix fragments across the 32 threads of a warp. No single lane owns a whole input or output tile. All participating lanes must execute consistently; divergent participation is invalid for synchronous operations.

Hierarchical tiling

Large matrices are tiled at several levels:

- Grid: complete GEMM
- Thread block/CTA: shared-memory working set
- Warp or warp group: one or more output tiles
- Instruction: hardware-supported MMA shape

Good tile sizes balance reuse, occupancy, register count, shared memory, and edge handling.

Data movement pipeline

Global-memory bandwidth and latency can starve Tensor Cores. Optimized kernels use coalescing, shared-memory staging, double buffering, and asynchronous copy mechanisms available on the target architecture to overlap loading with computation.

Register pressure and occupancy

Accumulator fragments occupy registers across many K iterations. Larger tiles increase reuse but also increase registers per thread, which can reduce active warps. Maximum occupancy is not the goal; enough residency to hide latency while retaining reuse is.

Evolution and instruction scope

Tensor Cores first exposed warp-level matrix operations and later architectures expanded types, shapes, sparsity, data movement, and warp-group/asynchronous mechanisms. Code must target documented compute capabilities; instruction availability is not universal.

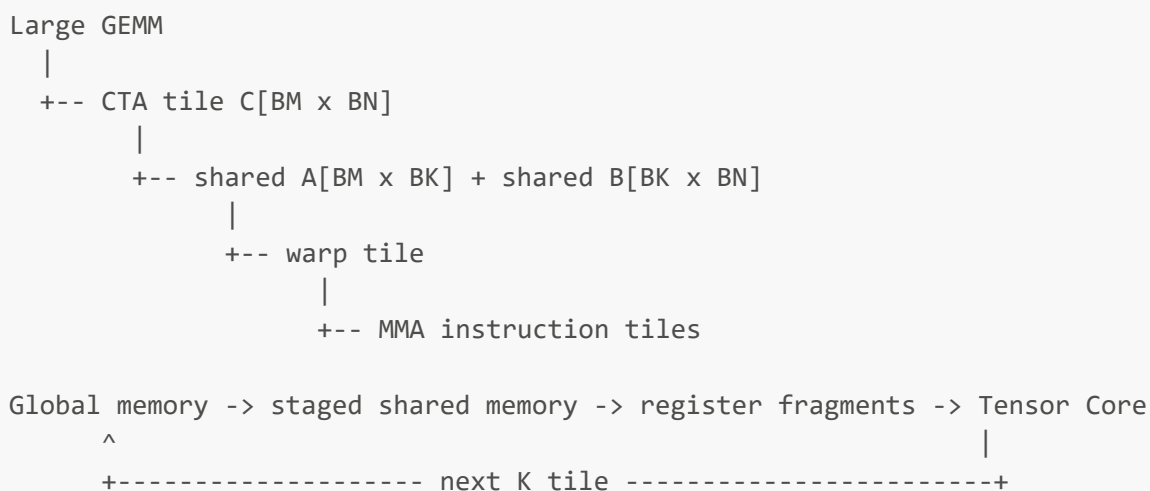
4. Real-World Example

For transformer inference, the projection $Y = XW + b$ is a GEMM followed by bias. A cuBLASLt or TensorRT kernel can:

1. Read FP16/BF16/INT8 tiles.
2. Run Tensor Core MMA in a pipelined K loop.
3. Keep sums in FP32 or INT32 accumulators.
4. Apply bias and activation in the epilogue.
5. Store the requested output type once.

Fusing the epilogue avoids an intermediate matrix write and another kernel launch.

5. Diagrams / Mental Models



Limiters	Symptoms	Typical responses
Global memory	Low arithmetic intensity	Increase tile reuse/fusion
Shared-memory conflicts	Serialization	Change layout/padding/swizzle
Register pressure	Low residency/spills	Reduce tile/accumulator count
Instruction dependency	Tensor pipeline stalls	Multiple independent accumulators/pipelining
Small/edge problem	Setup and masked-work overhead	Library heuristic or separate remainder kernel

6. Common Interview Questions

1. **What does a Tensor Core do?** It performs small matrix multiply-accumulate operations cooperatively at high throughput. Mistake: calling it a standalone CPU-like core running one thread.
2. **What is the basic operation?** $D=A*B+C$ on fixed-size tiles. Expected: types/shapes are architecture-dependent. Mistake: assuming one universal 4×4 shape.
3. **Why are Tensor Cores fast?** Specialized datapaths perform many multiply-accumulates per instruction with reduced-precision operands and wide accumulators. Mistake: ignoring data supply.
4. **Does one CUDA thread invoke a whole Tensor Core tile?** The instruction is normally warp- or warp-group-cooperative; fragment elements are distributed. Mistake: assigning the whole tile to lane 0.
5. **What precision do Tensor Cores use?** Multiple formats are supported depending on architecture, including variants of FP16, BF16, TF32, INT8, and newer types, with defined accumulator choices. Mistake: saying "only FP16."
6. **Why use shared memory?** To reuse global data across multiple MMA operations and arrange efficient fragment loads. Mistake: assuming shared memory is automatically faster regardless of access pattern.
7. **What is arithmetic intensity?** Operations performed per byte moved. Tensor Cores need high reuse/intensity to stay compute-bound. Mistake: equating high FLOPS with high application performance.
8. **Why can large tiles hurt?** They increase registers/shared memory, reduce occupancy, and complicate edges. Mistake: always choosing the biggest tile.
9. **How do you verify Tensor Core use?** Profile kernels/instruction metrics, inspect generated code where necessary, and confirm the selected library algorithm. Mistake: infer use solely from FP16 input.
10. **When should you write a custom Tensor Core kernel?** When a library cannot efficiently express required fusion, layout, or specialized operation and profiling shows the opportunity. Mistake: replacing tuned GEMM for learning-code performance.

7. Deep-Dive Questions

1. **How do double buffering and asynchronous copies help?** While one shared-memory stage feeds MMA, another stage loads the next global tile. Correct synchronization prevents overwrite and exposes overlap between memory and compute.
2. **What limits the K loop?** Operand load bandwidth, shared-memory conflicts, register dependencies, instruction issue rate, and pipeline latency. Multiple accumulator tiles can create independent work but consume more registers.

3. **Why are fragment layouts opaque?** Hardware mappings can differ across architectures. Public APIs deliberately prevent portable assumptions about which lane/register owns a logical element.
4. **What is structured sparsity acceleration?** Supported hardware/instructions exploit a specified sparse pattern to reduce effective multiply work. The data must obey metadata/layout rules; arbitrary zeros do not automatically accelerate.
5. **How would you reason about a roofline?** Compute arithmetic intensity for the tiled operation, compare achievable memory bandwidth and Tensor Core throughput, then profile. If the bandwidth roof is lower, more compute units will not help without reuse or fewer bytes.

8. Comparison Tables

Aspect	CUDA cores	Tensor Cores
Operation granularity	Scalar/vector arithmetic	Matrix tile MMA
Thread scope	Per-thread instruction semantics	Cooperative warp/warp-group semantics
Flexibility	General arithmetic/control	Constrained shapes/types/layouts
Best workloads	Element-wise, control-heavy, general kernels	Dense/sparse supported matrix operations
Main optimization	Coalescing, occupancy, instruction mix	Same, plus tile reuse and Tensor pipeline feed
Interface level	Advantage	Cost
cuBLAS/cuBLASLt/cuDNN	Tuned, portable, maintained	Limited to exposed operations/epilogues
CUTLASS/templates	Customizable kernel building blocks	More compile and design complexity
WMMA C++ API	Readable warp-matrix abstraction	Limited shapes/types, opaque fragments
PTX MMA/WGMMA	Precise instruction control	Architecture coupling and register-layout burden

9. Common Mistakes

- Focusing on the MMA instruction and ignoring memory movement
- Assuming low precision automatically selects Tensor Cores
- Using divergent control flow around warp-synchronous matrix operations
- Overfilling registers/shared memory and destroying useful residency
- Assuming one architecture's shape/layout rules apply to every GPU
- Benchmarking only perfect multiples and ignoring real edge tiles

10. Edge Cases / Special Cases

- Matrix dimensions not divisible by kernel tiles require padding, predication, or cleanup paths.
- Misaligned pointers/strides can prevent vectorized or supported loads.

- Shared-memory bank conflicts can reduce operand-delivery rate.
- Numerical results may vary with tile shape and reduction order.
- Some operations accept low-precision inputs but a different accumulator/output type is mandatory.
- Newer warp-group asynchronous instructions have different synchronization and lifetime rules than classic warp-synchronous MMA.

11. How to Explain in Interview

“Tensor Cores are specialized matrix multiply-accumulate units. Warps or warp groups feed fixed-shape operand tiles, execute $D=A*B+C$, and commonly use a wider accumulator. High performance depends on hierarchical tiling, shared-memory reuse, a pipelined data path, and a fused epilogue—not just issuing an MMA instruction.”

12. Quick Revision Notes

- Basic primitive: tiled $D=A*B+C$.
- Cooperative execution; fragments are distributed across lanes.
- Types and shapes depend on compute capability.
- Feed path: global → shared → registers → MMA.
- Tile reuse raises arithmetic intensity.
- Trap: peak Tensor FLOPS mean nothing if memory or non-matrix work dominates.

13. Practice Tasks

1. Draw grid-, CTA-, warp-, and instruction-level tiles for one GEMM.
2. Calculate bytes and FLOPs for a tile and estimate arithmetic intensity.
3. Profile cuBLAS GEMM and identify Tensor Core and memory-pipeline metrics.
4. Compare separate bias/activation kernels with a fused library epilogue.
5. Sweep matrix shapes and explain poor performance on small or irregular cases.

14. Final Cheat Sheet

Item	Recall
Core definition	Specialized cooperative matrix multiply-accumulate datapath
Why it matters	Very high throughput for GEMM-like computation
Most asked	Tiling, warp cooperation, accumulation, arithmetic intensity
Key comparison	CUDA cores are general; Tensor Cores are constrained matrix engines
One-line answer	“Tensor Cores make tile MMA fast; the kernel’s real job is keeping those tiles fed.”

WMMA

1. Overview

Definition

WMMA means **Warp Matrix Multiply-Accumulate**. CUDA's C++ WMMA API, in namespace `nvcuda::wmma` from `<mma.h>`, gives a warp-level abstraction over supported Tensor Core matrix operations. Its core concepts are fragments and four common functions:

- `fragment` declares distributed warp-owned operands or accumulators.
- `load_matrix_sync` loads a tile into a fragment.
- `fill_fragment` initializes a fragment.
- `mma_sync` computes $D=A*B+C$.
- `store_matrix_sync` writes an accumulator fragment to memory.

Why it matters and where it is used

WMMA is useful for learning Tensor Core execution and for custom fused warp-level kernels whose shapes/types fit the API. Production plain GEMM should generally use cuBLAS/cuBLASLt; WMMA leaves boundary handling, tiling, staging, synchronization, and tuning to the programmer.

Why interviewers ask

It tests warp-uniform control, memory layout/stride constraints, opaque fragment ownership, tiling, and API-versus-instruction trade-offs.

2. Core Idea

Intuition and analogy

A `fragment` is a sealed puzzle box shared across a warp. The warp can load a logical matrix tile, ask the hardware to multiply boxes, and store the result. You may apply a uniform element-wise operation through documented fragment elements, but you must not assume which logical matrix element resides in a particular lane or register.

Small example

The following educational kernel computes one 16×16 output tile for $K=16$, assuming valid pointers and compatible row-major input/output storage. It intentionally shows the WMMA sequence, not a complete general GEMM:

```
#include <mma.h>
#include <cuda_fp16.h>

using namespace nvcuda;

__global__ void one_wmma_tile(const half* A, const half* B, float* C) {
    wmma::fragment<wmma::matrix_a, 16, 16, 16,
                    half, wmma::row_major> a;
    wmma::fragment<wmma::matrix_b, 16, 16, 16,
                    half, wmma::row_major> b;
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c;

    wmma::load_matrix_sync(a, A, 16);
    wmma::load_matrix_sync(b, B, 16);
```

```

wmma::fill_fragment(c, 0.0f);
wmma::mma_sync(c, a, b, c);
wmma::store_matrix_sync(C, c, 16, wmma::mem_row_major);
}

```

Launch one full warp. A real kernel adds block/warp tile indices, a loop over **K**, shared-memory staging, checks/padding, alpha/beta, and an epilogue.

Step by step

1. All 32 lanes create matching fragment declarations.
2. The warp collectively loads **A** and **B** fragments.
3. It initializes the accumulator uniformly.
4. Every lane calls **mma_sync** with identical template/control parameters.
5. The warp collectively stores the logical output matrix.

3. Important Subtopics

Fragment roles

matrix_a, **matrix_b**, and **accumulator** identify operand roles. Template parameters specify **m**, **n**, **k**, element type, and—where required—layout. Types and shapes must form a supported combination.

Warp-uniform execution

Synchronous WMMA calls must be reached by every active lane in the participating warp under uniform conditions. Branching so only some lanes call **mma_sync** causes undefined behavior or hangs/incorrectness.

Leading dimension and layout

The leading dimension is the stride between rows or columns according to layout, not the total element count. Pointers and strides must satisfy the documented type-specific alignment/multiplicity rules.

Opaque element mapping

The distribution of logical matrix elements across lanes and **fragment.x[]** is unspecified. Store to memory before addressing logical (**row**, **column**) elements unless applying the same element-wise transform to every fragment element.

Multi-tile GEMM

A useful WMMA GEMM nests CTA/warp tiling around a **K** loop. Each iteration loads another **A/B** tile and calls **mma_sync** with the same accumulator. Shared memory is used for reuse and layout adaptation.

4. Real-World Example

Suppose a custom layer computes $C = \text{ReLU}(A*B + \text{bias})$ and its layout is unsupported by a convenient library epilogue. A WMMA kernel can keep each **C** fragment in FP32 registers, add bias and apply ReLU, then store FP16 output. This avoids materializing **A*B**, launching a bias kernel, then launching ReLU. Before keeping this custom path, compare it with cuBLASLt/CUTLASS because those already support many epilogues.

5. Diagrams / Mental Models

```

all 32 lanes in one warp
|
load_matrix_sync(A) + load_matrix_sync(B)
|
v
[opaque fragments]
|
mma_sync      repeated across K tiles
|
v
[FP32 accumulator fragment]
|
store_matrix_sync

```

WMMA object/function	Purpose	Common trap
<code>fragment<matrix_a,...></code>	Distributed A tile	Wrong layout/type
<code>fragment<matrix_b,...></code>	Distributed B tile	Wrong leading dimension
<code>fragment<accumulator,...></code>	Distributed C/D tile	Assuming lane ownership
<code>load_matrix_sync</code>	Collective load	Divergent participation/alignment
<code>mma_sync</code>	Warp MMA	Mismatched shapes or partial warp
<code>store_matrix_sync</code>	Collective store	Wrong memory layout/stride

6. Common Interview Questions

- What is WMMA?** A CUDA C++ warp-level API for supported matrix multiply-accumulate operations. Mistake: calling it a standalone library GEMM.
- What is a fragment?** A warp-distributed register representation of a logical matrix tile. Mistake: assuming each thread holds a known row.
- Why does every lane participate?** The operation is warp-synchronous and jointly owns operands/results. Mistake: invoking it only from lane zero.
- What does `mma_sync` compute?** $D=A*B+C$, or in-place $C=A*B+C$, for matching fragments. Mistake: forgetting the accumulator input.
- What does `load_matrix_sync` require?** Uniform call parameters plus documented pointer alignment, layout, and leading-dimension constraints. Mistake: treating `ldm` as bytes without checking the API.
- Can you map `fragment.x[i]` to matrix coordinates?** Portably, no; mapping is unspecified. A uniform element-wise operation over all fragment elements is allowed. Mistake: hard-coding a lane-to-coordinate map.
- How do you handle matrices larger than one tile?** Assign output tiles to warps/blocks and loop through `K` tiles, accumulating each. Mistake: launching one warp per individual multiply element.
- How do you handle edge dimensions?** Pad inputs, use predicated staging into full tiles, or run a cleanup path. Mistake: out-of-bounds fragment loads.

9. **WMMA versus cuBLAS?** WMMA gives kernel-level control; cuBLAS gives a tuned full GEMM. Use cuBLAS by default. Mistake: assuming lower level is automatically faster.
10. **How do you know a WMMA combination is supported?** Check the CUDA Programming Guide for the target compute capability, types, layouts, and shapes; compile for and test the actual architecture. Mistake: relying on one blog's hardware generation.

7. Deep-Dive Questions

1. **Why is passing fragments between separately compiled architecture targets risky?** Fragment ABI/layout can differ by target. Avoid exposing fragments across incompatible compilation boundaries; load/store through memory or keep producer and consumer architecture-consistent.
2. **How can shared memory improve WMMA?** It coalesces global loads, enables reuse across warp tiles, and transforms layout. It must be synchronized at the block level before consumers read a completed stage.
3. **What does the optional saturation mode do?** For supported `mma_sync` use, saturation-to-finite can map positive/negative infinity to finite extremes and NaN to zero in the destination; it changes numerical semantics and should be explicit.
4. **Why might direct `fragment.x[]` transformation be valid for scaling but not bias by column?** Multiplying every held accumulator element by the same scalar does not require logical coordinates; column-dependent bias does.
5. **How would you pipeline a WMMA `K` loop?** Stage future operands while current fragments compute, alternate shared-memory buffers, synchronize ownership transitions, and balance the number of stages against shared-memory/register cost.

8. Comparison Tables

Aspect	WMMA	PTX <code>mma</code>	cuBLAS/cuBLASLt
Level	CUDA C++ warp abstraction	Low-level virtual ISA	Optimized library
Fragment mapping	Opaque	Explicit operand register packing	Hidden
Tuning burden	High	Very high	Low
Portability	Better than inline PTX, still capability-bound	Architecture/instruction-bound	Highest practical portability
Best use	Custom educational/fused tile kernel	Exact low-level control	Standard GEMM/batched GEMM/epilogues

9. Common Mistakes

- Launching fewer than a full participating warp
- Calling WMMA functions inside lane-divergent branches
- Misreading the leading dimension or memory layout
- Assuming `fragment.x[]` has a stable coordinate mapping
- Omitting a `K` loop and calling a one-tile demo a general GEMM

- Ignoring alignment, edge tiles, shared-memory races, and error checks

10. Edge Cases / Special Cases

- Shapes and types vary by compute capability; preview/sub-byte namespaces can have extra restrictions.
- Accumulator layout is omitted in its fragment declaration but specified when stored.
- Layout parameters and template arguments must be identical across participating lanes.
- Direct fragment data should not be treated as a stable ABI.
- A matrix can be mathematically row-major while an API/library expects a different physical convention; verify indexing with a known small case.
- `mma_sync` is a compute primitive, not a synchronization replacement for block-level shared-memory producer/consumer hazards.

11. How to Explain in Interview

"WMMA is CUDA's C++ warp-level Tensor Core API. A full warp collectively loads opaque matrix fragments, calls `mma_sync` for $D=A*B+C$, and stores the accumulator. I must keep calls warp-uniform, obey layout/alignment rules, and tile the full GEMM around the instruction. For normal GEMM I use cuBLAS first."

12. Quick Revision Notes

- Header/namespace: `<mma.h>`, `nvcuda::wmma`.
- Main API: fragment, load, fill, MMA, store.
- All lanes participate uniformly.
- Fragment logical mapping is unspecified.
- Add block/warp tiling and a `K` loop for real GEMM.
- Trap: WMMA is not automatically a complete or tuned GEMM.

13. Practice Tasks

1. Run the one-tile example on identity matrices and validate all 256 outputs.
2. Add a loop for `K > 16` while keeping an FP32 accumulator fragment.
3. Extend to multiple warp output tiles and handle boundary padding safely.
4. Add a uniform accumulator scale through `fragment.x[]` and explain why it is portable.
5. Compare performance/correctness with cuBLAS across square and irregular shapes.

14. Final Cheat Sheet

Item	Recall
Core definition	CUDA C++ warp-level matrix fragment API
Why it matters	Accessible custom Tensor Core programming
Most asked	Uniform participation, fragments, layouts, edge tiles
Key comparison	WMMA abstracts registers; PTX MMA exposes them; cuBLAS hides both

Item	Recall
One-line answer	"A warp loads fragments, performs $D=A*B+C$, and stores them—uniformly and under strict layout rules."

MMA Instructions

1. Overview

Definition

MMA instructions are low-level matrix multiply-accumulate operations exposed through PTX instruction families such as `mma.sync`, with additional families for specific tensor data or newer execution scopes. They specify instruction shape, operand layouts, input types, accumulator types, and register operands more explicitly than WMMA.

A conceptual PTX spelling is:

```
mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32
    {d-registers}, {a-registers}, {b-registers}, {c-registers};
```

This indicates a warp-synchronous aligned operation for an `m16n8k16` tile with row/column operand layouts, FP16 inputs, and FP32 accumulator/destination. Exact legal forms and register packing must be taken from the PTX ISA for the target version.

Why it matters and where it is used

PTX MMA is used in highly tuned libraries, compiler-generated kernels, CUTLASS-style implementations, research kernels, and custom operations requiring shapes or control not exposed conveniently by WMMA. Most application developers should call an optimized library rather than write inline PTX.

Why interviewers ask

It reveals understanding of ISA abstraction, warp semantics, lane/register packing, instruction shapes, portability, and the difference between PTX and native machine code (SASS).

2. Core Idea

Intuition and analogy

WMMA lets a courier deliver sealed boxes. PTX MMA asks you to pack each specific register parcel carried by each lane. You gain control over the exact instruction form, but a single packing, constraint, or architecture mistake corrupts the tile.

Small example and step-by-step model

For an `m16n8k16` operation:

1. The warp collectively represents an $A(16 \times 16)$ tile and a $B(16 \times 8)$ tile across lane registers.
2. Each lane places its assigned packed elements into registers exactly as documented.
3. Every lane executes the same `mma.sync` instruction.
4. Tensor Cores produce an $D(16 \times 8)$ tile distributed among destination registers.
5. Repeated instructions accumulate across larger K and N/M tiles.
6. Code maps distributed accumulator registers to an epilogue/store layout.

The logical matrix is warp-wide; inline assembly operands are per-thread register lists.

3. Important Subtopics

PTX versus SASS

PTX is NVIDIA's virtual ISA. The toolchain compiles PTX to architecture-specific machine instructions (SASS). A PTX `mma` form expresses desired semantics but the final instruction mapping belongs to the target compiler/architecture.

Instruction qualifiers

An MMA mnemonic encodes scope/synchronization, shape, operand layouts, destination/input types, and sometimes rounding, saturation, sparsity, or block-scaling variants. Qualifier order and legal combinations are exact; consult the PTX ISA rather than guessing.

Lane/register fragment mapping

Unlike WMMA's opaque fragments, PTX documents how each lane's registers correspond to tile elements for each instruction form. Correct pack/unpack code is essential and instruction-specific.

Inline PTX constraints

CUDA inline assembly must use correct operand constraints, register widths, braces, and `volatile`/dependency behavior where appropriate. C++ types alone do not guarantee correct packing; FP16 values are often packed into 32-bit registers for particular forms.

MMA, sparse MMA, and warp-group MMA

The PTX ISA contains several tensor-operation families. Warp-level `mma.sync`, sparse forms with metadata, and newer warp-group asynchronous operations differ in participant count, operand descriptors, synchronization, and supported architectures. Do not transfer assumptions between them.

4. Real-World Example

A fused attention kernel needs a particular MMA shape, shared-memory layout, and accumulator mapping so it can compute softmax-related transformations without storing the intermediate score matrix. A library author may use PTX MMA to control register fragments and the epilogue tightly. Application code should first use an optimized attention or GEMM library because the custom implementation must also solve masking, numerical stability, edges, scheduling, and architecture dispatch.

5. Diagrams / Mental Models

```

CUDA C++ source
|
+-- library / intrinsics / inline PTX
    |
    v
    PTX virtual ISA: mma.sync...
    |
    v ptxas/JIT for target SM
    SASS machine instructions
    |
    v
    Tensor Core hardware pipeline

logical matrix tile <--> documented lane/register fragments

```

Qualifier idea	Example meaning
<code>m16n8k16</code>	Logical output/tile dimensions M=16, N=8, reduction K=16
<code>row.col</code>	A and B operand layout declarations
<code>f16</code> inputs	Operand element formats
<code>f32</code> accumulator	C/D register element interpretation
<code>sync.aligned</code>	Warp convergence/alignment contract for that PTX form

6. Common Interview Questions

- What is an MMA instruction?** A cooperative fixed-shape matrix multiply-accumulate primitive, usually $D=A*B+C$. Mistake: describing a full arbitrary-size GEMM.
- What does `m16n8k16` mean?** The logical operation multiplies an $M \times K$ tile by a $K \times N$ tile to update an $M \times N$ tile. Mistake: treating it as thread-block dimensions.
- How is MMA different from WMMA?** MMA exposes exact PTX instruction/register forms; WMMA provides C++ fragment abstractions. Mistake: saying they are unrelated hardware.
- Is PTX the GPU's final machine code?** No. PTX is a virtual ISA compiled/JIT-compiled into architecture-specific SASS. Mistake: treating PTX latency/register mapping as permanently fixed native behavior.
- Who participates in `mma.sync`?** A converged warp according to the instruction contract. Mistake: executing it in a single lane.
- Why are operands register lists?** Each lane owns a documented piece of warp-wide matrix fragments. Mistake: assuming a pointer can be passed directly to `mma`.
- What does `row.col` indicate?** Logical layout declarations for A and B as defined by that instruction form. Mistake: confusing them with C++ array allocation alone.
- Why pack FP16 values?** Some instruction forms carry multiple low-precision elements in a 32-bit register. Exact packing is form-specific. Mistake: using arbitrary bit order.
- When use inline PTX?** Only when higher-level APIs cannot express or compile to the required instruction and profiling justifies architecture-specific code. Mistake: using it for ordinary GEMM.
- How do you verify generated MMA code?** Compile for the intended SM, inspect PTX/SASS as appropriate, profile tensor instruction execution, and validate numerical output. Mistake: checking only

that compilation succeeded.

7. Deep-Dive Questions

1. **What does `aligned` mean in the instruction contract?** It expresses required convergence/alignment of participating threads for the PTX operation, not merely pointer alignment. All threads required by the scope must execute the same instruction consistently.
2. **How do accumulator dependencies affect throughput?** Repeatedly updating one accumulator chain exposes instruction latency. Multiple independent accumulator fragments increase instruction-level parallelism but consume more registers.
3. **What must architecture dispatch handle?** Compute-capability availability, supported shapes/types, shared-memory features, instruction family, register mapping, and a fallback path. Fat binaries/templates can isolate variants.
4. **Why are sparse MMA instructions not a free speedup for matrices containing zeros?** They require a supported structured sparsity pattern plus encoded metadata and correct operand layout. Arbitrary sparsity does not satisfy the hardware contract.
5. **How do warp-group asynchronous MMA operations change kernel design?** More threads cooperate; operations can be asynchronous relative to issuing threads; operands may use descriptors and shared-memory mechanisms; explicit commit/wait and pipeline lifetime rules replace assumptions from classic synchronous warp MMA.

8. Comparison Tables

Aspect	Scalar FMA	PTX <code>mma.sync</code>	Warp-group async MMA
Logical granularity	One <code>a*b+c</code> per thread	Matrix tile per warp	Larger cooperative tile per warp group
Operand location	Scalar registers	Lane-packed fragment registers	Architecture-specific registers/descriptors/shared data
Synchronization	Thread-local	Warp-synchronous contract	Asynchronous group pipeline contract
Flexibility	General	Fixed shapes/types	Fixed architecture-specific shapes/types
Typical author	General CUDA programmer/compiler	Kernel/library specialist	Architecture-specific library specialist
Use this level	When		
cuBLAS/cuBLASLt	Standard or epilogue-fused GEMM		
WMMA	Educational/custom operation supported by its fragment API		
PTX MMA	Exact instruction/layout control has measured value		
SASS inspection	Verification/performance diagnosis, not normal source authoring		

9. Common Mistakes

- Guessing the PTX mnemonic or operand ordering
- Treating PTX as stable native machine code
- Copying lane mappings from a different shape/type
- Using wrong inline-assembly constraints or packed register types
- Diverging within the participating warp
- Omitting architecture guards and a fallback implementation
- Measuring a micro-instruction while ignoring the complete data pipeline

10. Edge Cases / Special Cases

- Legal instruction forms change by PTX ISA version and target SM.
- Accumulator input and output register counts/types can differ across shapes.
- Integer forms have signedness and saturation/wrap semantics that must be checked exactly.
- TF32 operands may use FP32 registers containing values rounded to TF32 precision.
- Inline PTX can inhibit compiler transformations or create hidden dependency issues if constraints are wrong.
- Register spills can make a theoretically superior tile much slower.
- New instruction families may require shared-memory address descriptors and explicit asynchronous completion.

11. How to Explain in Interview

"PTX MMA is the low-level warp-cooperative instruction interface behind Tensor Core matrix operations. The mnemonic fixes tile shape, layouts, types, and synchronization, while each lane supplies a documented register fragment. PTX is compiled to SASS, so I guard by architecture and prefer libraries or WMMA unless exact instruction control has measured value."

12. Quick Revision Notes

- MMA computes a fixed tile of $D=A*B+C$.
- $m/n/k$ describe logical matrix tile dimensions.
- Register fragments are distributed across lanes.
- PTX is virtual ISA; SASS is target machine code.
- Instruction forms are compute-capability dependent.
- Trap: lower-level control increases obligations, not guaranteed speed.

13. Practice Tasks

1. Read one PTX `mma.sync` form and draw every operand matrix dimension.
2. Map documented lane registers back to a logical output tile on paper.
3. Compile a WMMA example and inspect emitted PTX/SASS for matrix instructions.
4. Compare one accumulator chain with multiple independent accumulator tiles in profiler metrics.
5. Build a guarded architecture-specific microkernel plus a correct library/reference fallback.

14. Final Cheat Sheet

Item	Recall
Core definition	Low-level fixed-shape cooperative matrix multiply-accumulate instruction
Why it matters	Exact control over Tensor Core instruction and register fragments
Most asked	m/n/k, PTX vs SASS, lane mapping, WMMA comparison
Key comparison	WMMA hides fragment mapping; MMA exposes it; libraries hide and tune the whole kernel
One-line answer	"MMA is the instruction primitive; GEMM performance comes from building the right memory and tiling pipeline around it."

Choosing Optimized Libraries: Interview Decision Guide

The nine topics above converge on one practical rule: select the highest-level interface that fully expresses the operation and validate its numerical policy.

Need	First choice	Move lower only when...
Dense/batched GEMM	cuBLAS or cuBLASLt	Required layout/fusion is unsupported or measured performance is inadequate
Convolution/normalization/DNN primitive	cuDNN	A genuinely custom operator cannot be expressed/fused
Optimized inference graph	TensorRT	A plugin/custom kernel is needed for an unsupported operator
Custom composable GEMM kernel	CUTLASS-style building blocks	Template capabilities still cannot express the algorithm
Educational/simple warp tile	WMMA	Exact instruction form/register mapping is required
Architecture-specific microkernel	PTX MMA/WGMMA	There is measured benefit worth ongoing maintenance

Library checklist

1. **Correctness contract:** decide allowed input, compute, accumulator, and output precision.
2. **Hardware support:** query compute capability and library support instead of assuming.
3. **Shape/layout:** use valid leading dimensions, contiguous/coalesced layouts, and padding where measured useful.
4. **Fusion:** prefer supported bias/activation/scale epilogues to extra memory round trips.
5. **Workspace:** allow an appropriate workspace budget so the library can choose faster algorithms.
6. **Benchmark:** warm up, use CUDA events, test production shapes/batches, and include conversions.

7. **Profile:** confirm Tensor Core instructions and diagnose memory, occupancy, or fallback bottlenecks.
8. **Validate:** compare with a higher-precision reference and end-to-end quality metric.
9. **Record policy:** make TF32/precision/determinism choices explicit rather than relying on defaults.

Cross-topic final comparison

Question	FP32	FP16	BF16	TF32	INT8
Bytes stored/value	4	2	2	Usually 4	1
Main advantage	Reliable general baseline	Better 16-bit precision	Better 16-bit range	Easy fast FP32 matrix path	Highest compression/inference throughput
Common wide accumulator	FP32	FP32	FP32	FP32	INT32
Main numerical risk	Rounding/reduction error	Under/overflow	Coarse rounding	Reduced multiply precision	Rounding + clipping + scale error
Key support mechanism	cuBLAS SGEMM/general CUDA	AMP + Tensor Cores	AMP + Tensor Cores	Library math policy	PTQ/QAT + TensorRT/runtime

References

- [CUDA Programming Guide: C/C++ language extensions and WMMA](#)
- [CUDA Programming Guide: floating-point computation](#)
- [PTX ISA: matrix and tensor instructions](#)
- [CUDA Math API: half precision](#)
- [NVIDIA mixed-precision training guide](#)
- [TensorRT accuracy and quantization considerations](#)

Hardware support, legal instruction forms, and library defaults evolve. For implementation work, check the documentation matching the installed CUDA Toolkit, target GPU compute capability, framework, and inference runtime.